

CLAUDE.md — CAIRN

Project: Cairn — A Discipline-First Trading Journal **Version:** 1.0 Specification **Author of spec:** Trading coach collaboration **App creator credit:** Designed & built by Jai Akash
Intended build tool: Claude Code (Anthropic)

0. HOW TO USE THIS DOCUMENT

This is the master specification for Cairn. It is written as one comprehensive document for approval, but it is explicitly structured for clean splitting into a `/docs` folder. Each major section below is marked with a `[SPLIT → filename.md]` tag indicating which sub-document it should become once split.

Recommended workflow:

1. Read this entire document once to understand the whole system.
2. Split into `/docs/*.md` sub-files using the split markers.
3. Keep a slim root `CLAUDE.md` that contains only Sections 1, 2, 3, and a pointer map to the sub-files.
4. When working on any feature, load only the relevant sub-files into context.

This document is the single source of truth. When it conflicts with any assumption, prior message, or intuition — follow this document.

1. FOUNDING DOCUMENT — WHY CAIRN EXISTS

`[SPLIT → docs/philosophy.md]`

1.1 The Problem This App Solves

A cairn is a stack of stones placed on a trail to mark the way for those who come after. Each stone is deliberate. Each stone stays where it's placed. A cairn is built by people who have walked the path and want to make sure they — and others — can find it again.

This app exists because someone walked off the path roughly 45-50 times and wants to build the cairn that keeps them on it.

The user who commissioned this app has:

- Studied ICT (Inner Circle Trader) methodology for over a year.
- Attempted roughly 45-50 funded prop-firm challenges.
- Cleared Phase 1 approximately 70% of the time.
- Cleared Phase 2 only 7-10% of the time.
- Diagnosed their own problem as primarily discipline and process, not knowledge.

After structured assessment, the diagnosis was confirmed: **this is ~20% knowledge gap, ~80% discipline and process gap.** The trader knows how to take a good trade. The trader does not yet have the system that prevents them from taking bad ones.

Every feature in this app exists to solve one specific pattern: **"I know the rule. I break the rule. I lose. I promise to follow the rule. I break it again."**

1.2 The Design Philosophy

Most trading journals are **records of what you did.** Cairn is a **system that changes what you do.**

This one sentence shapes every decision in this spec. When there is a choice between:

- A nicer report vs. a rule that prevents a bad trade → prevent the bad trade.
- A faster log entry vs. a field that forces self-awareness → force the self-awareness.

- A generic analytic vs. an ICT-specific analytic → go ICT-specific.
- A friendly nudge vs. a hard block when a rule is breaking → hard block.

The app is an **external discipline layer**. The user has proven (45-50 times) that willpower alone does not work during live markets. Cairn replaces willpower with structure.

1.3 The Three Jobs Cairn Must Do

1. **Prevent rule violations in real time, before the click.** Not detect them afterwards. Prevent them.
2. **Capture complete, honest data on every trade.** With no escape hatch for laziness or tilt.
3. **Transform that data into insights that change behavior week over week.**

If a feature doesn't serve one of these three jobs, it doesn't belong in v1.

1.4 What Cairn Is Not

- Cairn is not a charting tool. The user trades on TradingView and MT5.
- Cairn is not a social platform. There is no sharing, no feeds, no comparisons to other traders.
- Cairn is not a coach. It doesn't give advice during trades. It enforces the user's own pre-committed rules.
- Cairn is not tied to any specific prop firm. It is neutral and configurable.
- Cairn is not cloud-first. It is a private, local, personal tool.

1.5 Voice and Tone (For UI Copy)

Cairn speaks the way a respected mentor speaks: direct, calm, and honest. Never cutesy. Never alarmist. Never patronizing. Examples of correct voice:

- ❌ "Oops! Looks like you've hit your daily limit! 🎉"
 - ✅ "Daily loss limit reached. Session closed."
 - ❌ "Great job on that trade, champion!"
 - ✅ "Trade closed. +2.3R. Rules: clean."
 - ❌ "Are you sure you want to move your stop loss?"
 - ✅ "You are moving SL against you. This violates Rule 4 on this account. Proceed anyway?"
-

2. CORE PRINCIPLES — NON-NEGOTIABLE

[SPLIT → Keep in root CLAUDE.md]

These principles override any other consideration in the app. If an implementation decision seems to contradict one of these, stop and re-read.

2.1 Prevention Over Detection

The app must block rule-breaking actions in real time wherever possible. Post-hoc analytics are secondary. If a rule can be checked before the trade is placed, it must be.

2.2 Friction in the Right Places

Logging a trade must be fast (under ~90 seconds total round-trip). Breaking a rule must be slow (friction, warnings, typed confirmations). The app should be easy to use correctly and hard to use incorrectly.

2.3 Honesty Forcing Functions

Fields that require self-awareness (invalidation, emotional state, rules-broken) must be structured so that skipping them is either impossible or explicitly acknowledged. Never

default these to "N/A."

2.4 Local-First, Privacy-First

All data is the user's. It lives on their machine. No telemetry. No analytics sent anywhere. No account required. No network calls except for optional cloud-folder backup (which uses existing desktop sync tools, not APIs).

2.5 Data Integrity is Sacred

This is a financial app. Money-adjacent calculations must be correct to the cent/pip. All arithmetic uses appropriate decimal types, never floats. All P&L math is tested. All database writes are transactional.

2.6 Extensibility Without Rework

The user will add v2 features and MT5/cTrader integration later. Every architectural decision in v1 must accommodate v2 without requiring rewrites. Data model, component structure, and state management are all designed to be added to, never ripped up.

2.7 The App is a Cockpit, Not a Notebook

The trader should trade **through** Cairn — lot size calculator, rule-checker, target validator — not log in Cairn **after** trading elsewhere. This is the difference between a journal traders skip and a journal that becomes indispensable.

2.8 Aesthetic is Functional

Good design is a feature, not a polish layer. A calm, beautiful, deliberate UI reinforces the discipline the app is teaching. A cluttered or garish UI undermines it.

2.9 Never Mention Specific Prop Firms in UI

The app is prop-firm-agnostic. Prop firms are configured as generic "firms" with configurable rule sets. No firm name is hardcoded or referenced in user-visible text.

2.10 No Reference to the User's Backstory in the App

The founding document above exists for context to whoever is building the app. The user's history (45-50 accounts, etc.) must never appear in the UI, error messages, onboarding, or any user-visible text. The app speaks to a trader, not to *this* trader's history.

3. TECH STACK & ARCHITECTURE

[SPLIT → Keep in root CLAUDE.md]

3.1 Stack

Layer	Choice	Rationale
Runtime	Electron (latest stable)	Desktop app, Win/Mac/Linux, mature ecosystem
Renderer	React 18+ with TypeScript (strict mode)	Type safety for financial math
Build tool	Vite (via electron-vite)	Fast HMR, modern defaults
Styling	Tailwind CSS	Rapid styling, design-token friendly
Components	shadcn/ui (customized) + custom	Polished baseline, full control
Animation	Framer Motion	Physics-based motion, declarative
State	Zustand	Simple, boilerplate-free, TypeScript-native
Database	SQLite via better-sqlite3	Fast, synchronous, zero-config

Layer	Choice	Rationale
ORM / Migrations	Drizzle ORM	Type-safe queries, migration system
Charts	Recharts	React-native, customizable, good defaults
Forms	React Hook Form + Zod	Validation + type inference
Date/Time	date-fns + date-fns-tz	Timezone-critical for killzones
Icons	Lucide React	Clean, consistent
Fonts	Inter (UI), JetBrains Mono (numbers)	Pro-trader convention for numbers
Packaging	electron-builder	NSIS installer, DMG, AppImage

3.2 Package Manager & Node Version

- Use `pnpm` (faster, disk-efficient, strict).
- Node version pinned via `.nvmrc` (latest LTS at time of build).
- `engines` field in `package.json` enforces Node version.

3.3 File Structure

```
cairn/  
├─ CLAUDE.md           # Root spec (slim version after split)  
├─ docs/  
│   ├─ philosophy.md  
│   ├─ design-system.md  
│   └─ data-model.md  
└─
```

```
| |─ features-v1.md
| |─ features-v2.md
| |─ rules-engine.md
| |─ analytics.md
| |─ ui-flows.md
| |─ customization.md
| |─ integrations-future.md
| |─ testing.md
| |─ conventions.md
|─ electron/                # Main process
| |─ main.ts                # App entry, window management
| |─ preload.ts             # Context bridge
| |─ ipc/                   # IPC handlers grouped by domain
| | |─ trades.ts
| | |─ accounts.ts
| | |─ rules.ts
| | |─ backup.ts
| | |─ settings.ts
| |─ db/
| | |─ index.ts             # DB connection singleton
| | |─ schema.ts           # Drizzle schema
| | |─ migrations/         # Migration files, versioned
| | | |─ seed.ts           # Default data seed
| |─ services/              # Business logic (not UI)
| | |─ rules-engine.ts
| | |─ pnl-calculator.ts
| | |─ backup-service.ts
| | |─ import-adapters/    # v2 MT5/cTrader adapters slot here
| | | |─ manual.ts         # v1 "adapter" (just passthrough)
| | | |─ export-service.ts
| |─ utils/
|─ src/                     # Renderer (React app)
```

```
| |─ main.tsx
| |─ App.tsx
| |─ router.tsx
| |─ assets/
| |   |─ fonts/
| |   └─ icons/
| |─ components/
| |   |─ ui/                # shadcn/ui customized primitives
| |   |─ layout/
| |       |─ Sidebar.tsx
| |       |─ TopBar.tsx
| |       └─ Shell.tsx
| |   |─ dashboard/
| |   |─ trade-entry/
| |   |─ trade-log/
| |   |─ analytics/
| |   |─ accounts/
| |   |─ settings/
| |   └─ shared/
| |─ features/              # Feature modules (page + hooks + logic)
| |   |─ dashboard/
| |   |─ pre-trade/
| |   |─ post-trade/
| |   |─ analytics/
| |   |─ accounts/
| |   |─ settings/
| |   └─ review/
| |─ stores/                # Zustand stores
| |   |─ session-store.ts    # Current session bias, trades today
| |   |─ settings-store.ts
| |   └─ ui-store.ts
| |─ hooks/
```

```

|   └─ lib/
|     └─ ipc.ts           # Typed IPC wrapper
|     └─ formatters.ts    # Money, pips, percentages
|     └─ calculators.ts   # Lot size, RR, pip value
|       └─ cn.ts          # Tailwind class utility
|   └─ styles/
|     └─ globals.css
|   └─ types/
|     └─ index.ts         # Shared types
└─ shared/                # Types shared between main and renderer
    └─ types/
└─ tests/
    └─ unit/
    └─ integration/
    └─ e2e/
└─ scripts/               # Build, migrate, seed scripts
└─ .nvmrc
└─ package.json
└─ pnpm-lock.yaml
└─ tsconfig.json
└─ tsconfig.node.json
└─ vite.config.ts
└─ electron-builder.yml
└─ tailwind.config.ts
└─ postcss.config.js
└─ .env.example

```

3.4 Process Architecture

- **Main process** owns: database, file system, OS integration, backup service, window management.
- **Renderer process** owns: UI, transient UI state, user interactions.

- **Communication:** typed IPC only. No `remote` module. Context isolation enabled.
- **Preload script** exposes a narrow, typed API surface to the renderer.

3.5 Typed IPC Pattern

Every IPC handler:

1. Has a defined input Zod schema and output type.
 2. Is registered in a handler map in `electron/ipc/index.ts`.
 3. Is accessed from the renderer through a typed wrapper in `src/lib/ipc.ts`.
 4. Returns `{ ok: true, data } | { ok: false, error }` — never throws across the bridge.
-

4. DESIGN SYSTEM

`[SPLIT → docs/design-system.md]`

4.1 Design Direction

"Bloomberg Terminal meets Linear meets a Zen garden."

Professional but calm. Information-dense but spacious. Every pixel earns its place. The app should feel like a tool owned by a serious person — not a flashy retail product.

4.2 Color Palette

Philosophy: Two modes. Both are rich, deliberate, and distinct from common trading-app palettes (no generic purple/cyan, no Robinhood green, no Bloomberg orange-on-black cliché).

Dark mode — "Graphite & Citrus"

- Base background:  #0E1012 (warm graphite, not pure black)
- Surface (cards):  #16191D
- Surface elevated (modals):  #1C2025
- Border subtle:  #23272E
- Border strong:  #2E333B
- Text primary:  #E8EAED
- Text secondary:  #9BA1A9
- Text muted:  #5F656E
- **Accent A — Signal (wins, positive, active):**  #B4E048 (electric lime)
- **Accent B — Process (discipline, rules, gold-standard):**  #D4A24C (muted amber-gold)
- Warning:  #E8A33D
- Danger:  #E25C5C (desaturated, not fire-engine red)
- Info:  #6B9FFF

Light mode — "Bone & Forest"

- Base background:  #F7F5F0 (warm bone, not cold white)
- Surface:  #FFFFFF
- Surface elevated:  #FFFFFF with stronger shadow
- Border subtle:  #E8E4DC
- Border strong:  #D4CFC4
- Text primary:  #1A1D21
- Text secondary:  #4F5560
- Text muted:  #8B8F96

- **Accent A — Signal:** ■ #2F6B3A (deep forest green)
- **Accent B — Process:** ■ #A0762B (antique bronze)
- **Warning:** ■ #C47B15
- **Danger:** ■ #B64545
- **Info:** ■ #2F5ABF

Semantic pairs (used consistently everywhere):

- Win/profit → Accent A
- Clean rules followed → Accent B
- Loss → Danger (muted)
- Rule broken → Danger with a specific "cracked" treatment (see §4.7)

Gradient usage: Extremely sparingly. Only on the Discipline Ring (§4.9) and the main dashboard hero number. Never on buttons or cards.

4.3 Typography

Fonts:

- **Inter** — UI, body, labels (variable font, weights 400/500/600/700)
- **JetBrains Mono** — all numbers, prices, pip values, timestamps, account IDs (weights 400/500/600)

Scale (dark/light mode same):

- display-xl — 56px / 64 / -0.02em (rarely used, dashboard hero)
- display-lg — 40px / 48 / -0.02em
- display — 32px / 40 / -0.01em
- h1 — 24px / 32 / -0.01em

- `h2` — 20px / 28
- `h3` — 17px / 24
- `body-lg` — 16px / 24
- `body` — 14px / 20
- `body-sm` — 13px / 18
- `caption` — 12px / 16 / 0.01em (labels, captions)
- `micro` — 11px / 14 / 0.03em uppercase (badges, tags)

Number formatting rules:

- Always JetBrains Mono for numbers.
- Tabular figures enabled (`font-variant-numeric: tabular-nums`).
- Currency prefix (no trailing). E.g., `$5,000.00` not `5,000.00 USD`.
- Pips with 1 decimal for FX, e.g., `24.3 pips`.
- Percentages with 2 decimals, e.g., `-0.87%`.
- R-multiples with 2 decimals, explicit sign, e.g., `+2.34R`, `-1.00R`.

4.4 Spacing & Layout

- **Base unit:** 4px. Everything is a multiple (4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96).
- **Content max-width:** 1440px, centered on wider displays.
- **Sidebar width:** 240px, collapsed 64px.
- **Card padding:** 20px default, 24px for primary cards, 16px for compact.
- **Card radius:** 14px default, 10px for compact elements, 20px for hero elements.
- **Grid gap:** 16px default, 12px compact, 24px generous.

4.5 Shadows & Depth

Dark mode: Shadows are barely visible (`rgba(0,0,0,0.4)` small offsets). Depth is conveyed by background lightness steps, not shadows. Optional inner glow on focused cards using border + subtle accent-color (`box-shadow`) at 8% opacity.

Light mode: Shadows are the primary depth mechanic.

- `shadow-sm`: `0 1px 2px rgba(15, 20, 25, 0.04), 0 1px 3px rgba(15, 20, 25, 0.06)`
- `shadow-md`: `0 4px 8px rgba(15, 20, 25, 0.04), 0 2px 4px rgba(15, 20, 25, 0.06)`
- `shadow-lg`: `0 12px 24px rgba(15, 20, 25, 0.08), 0 4px 8px rgba(15, 20, 25, 0.06)`
- `shadow-xl`: `0 24px 48px rgba(15, 20, 25, 0.12), 0 8px 16px rgba(15, 20, 25, 0.08)`

4.6 Motion Language

Principles:

1. Motion is physical. Use spring physics (Framer Motion (`spring`)), not linear easing for entrances.
2. Motion is purposeful. Every animation communicates state or hierarchy.
3. Motion is restrained. Never loop. Never decorate.
4. Motion respects reduced motion preference (`prefers-reduced-motion: reduce`) → transitions become instant, no spring overshoot).

Spring defaults:

- Default: (stiffness: 260, damping: 26)
- Bouncy (celebrations, rare): (stiffness: 400, damping: 20)
- Settled (layouts): (stiffness: 180, damping: 30)

Standard durations (for non-spring):

- Instant: 80ms
- Short: 160ms
- Base: 240ms
- Long: 400ms
- Hero: 600ms

Specific motion moments:

Moment	Motion
Page transition	Cross-fade 160ms, no slide
Card entrance (grid reveal)	Stagger 40ms, spring from y+12 opacity 0
Number count-up	800ms ease-out, dashboard numbers only
Win trade logged	Accent A soft pulse 1x, 400ms
Loss trade logged	Danger soft pulse 1x, 400ms
Rule violation attempted	Shake horizontally 4px, 3 oscillations, 300ms
Rule gate unlock (cleared)	Checkmark draw-on 240ms + soft glow
Trade save success	Toast slide up spring, auto-dismiss 2.4s
Modal open	Scale 0.96→1 + opacity 0→1, spring
Modal close	Scale 1→0.98 + opacity 1→0, 120ms ease-in
Sidebar icon hover	Scale 1.08, 120ms
Button press	Scale 0.97, 80ms
Tab change	Underline slides to new tab, spring
Chart entrance	Path draw-on 600ms ease-out
Discipline Ring fill/crack	See §4.9

4.7 Rule Violation Visual Treatment

When a rule is violated (either attempted or logged on a past trade):

- The field/card/trade row gets a "**cracked**" **visual** — a thin danger-colored border with 2-3 asymmetrical "crack" lines drawn across it using SVG path animations.
- Accompanying subtle red tint on background (2% opacity).
- Tooltip/label states which rule was violated in plain English.
- This visual is never cutesy. It looks like a structural flaw.

4.8 Iconography

- Lucide React throughout. Stroke width 1.5 (slightly thinner than default for elegance).
- Size scale: 14, 16, 18, 20, 24, 28.
- Color inherits from `currentColor`; never colored icons unless semantic (e.g., accent A for wins).
- Avoid emoji anywhere in the UI. Exceptions: none.

4.9 Signature Element — The Discipline Ring

A circular progress ring displayed prominently on the dashboard. It is the app's signature visual.

Mechanics:

- Represents **Rule Adherence Score** for the current period (default: last 20 trades, configurable).
- 0% to 100% fill, animated.
- Fill color:
 - 95-100% → Accent A (signal green/forest green) with soft outer glow

- 80-94% → Accent B (amber-gold/bronze)
- 60-79% → Warning (muted orange)
- Below 60% → Danger, with visible **cracks** in the ring (SVG paths breaking through the stroke)
- Center displays the number, JetBrains Mono, `display-lg`.
- Below the number: a caption stating the calculation basis (e.g., "Last 20 trades").
- On hover: transitions to show a breakdown (which rules were broken most).
- When a rule is broken in real time: the ring briefly shakes, a crack appears, and the score updates with a spring animation.

This ring is what the user sees first on the dashboard. It is the moral compass of the app.

4.10 Component Aesthetic Baseline

Cards:

- 14px radius, surface color, 1px subtle border.
- No drop shadows in dark mode; depth via color.
- Hover: subtle border brightening (no transform, no lift).
- Focus-within: accent border, soft glow.

Buttons:

- Primary: filled with Accent A (signal), text in base-bg color. Hover: brightness up 6%. Active: scale 0.97.
- Secondary: 1px border, transparent bg, text primary. Hover: subtle bg tint.
- Ghost: no border, no bg, text secondary. Hover: subtle bg tint.
- Destructive: danger-colored filled. Used only for actual destructive actions.

- Disabled: 40% opacity, cursor not-allowed, no hover state.
- Sizes: sm (32px), md (38px), lg (44px). Padding scales accordingly.
- Corner radius: 10px default.

Inputs:

- Surface elevated bg, 1px subtle border, 10px radius.
- Focus: Accent A border, soft glow (2px, 12% opacity).
- Error: Danger border, small inline error text below.
- Numeric inputs: JetBrains Mono, right-aligned, with visible unit badges (e.g., "pips", "%", "\$").
- Label above, small caption hints below.

Badges:

- Small, 6px radius, caption-size.
- Variants: default, success (A), process (B), warning, danger, info, neutral.
- Solid or outlined; outlined preferred for frequency density.

Tables:

- Zero-border rows, 1px horizontal divider only.
- Sticky header with subtle tint.
- Row hover: 2% accent tint.
- Monospace for any numeric column.
- Sortable columns show sort indicator on hover.
- Row actions revealed on hover (right-aligned icon buttons).

Modals & Dialogs:

- Center screen, max-width 560px unless content-specific.
- Scale+fade entrance.
- Backdrop: base-bg at 72% with 8px blur.
- Trap focus, close on ESC, close on backdrop click (except for destructive-confirm dialogs).

Toasts:

- Bottom-right stack, max 3 visible.
- Slide-up spring entrance.
- 2.4s auto-dismiss default, 6s for errors, sticky for rule violations.
- Include undo action where applicable.

4.11 Dark/Light Mode Toggle

- Toggle in sidebar bottom.
- Sun/moon icon with spring rotation.
- Theme switch is instant, no flicker (apply class to `html` before paint).
- Stored in settings (localStorage + SQLite).
- Respects OS preference on first launch, then honors user choice.

4.12 Attribution

In the sidebar bottom-left corner, below the theme toggle:

- Text: `Designed & built by Jai Akash`
- Font: Inter, 11px, text-muted, letter-spacing 0.02em.
- Hover: opacity 1 (from 0.4), 160ms ease.

- Always present. Never referenced anywhere else in the app.
-

5. DATA MODEL

[SPLIT → docs/data-model.md]

5.1 Schema Overview

All tables use:

- `id` — UUID v7 (time-ordered) as TEXT primary key.
- `created_at` / `updated_at` — UNIX millisecond timestamps as INTEGER.
- `deleted_at` — soft delete timestamp, nullable.

All monetary values stored as INTEGER (minor units — cents/pips $\times 10$) to avoid floating-point errors. Converted to display format via formatters.

All enums stored as TEXT with CHECK constraints. Enum values defined once in `shared/types/enums.ts`.

5.2 Tables

5.2.1 `settings`

Key-value store for global app settings.

```
key          TEXT PK
value        TEXT (JSON)
updated_at   INTEGER
```

Keys include: `theme`, `default_pair`, `default_risk_pct`, `backup_folder_path`, `backup_schedule`, `timezone`, `week_starts_on`, `onboarding_completed`.

5.2.2 `prop_firms`

User-configurable firms. Pre-seeded with "Custom" entry. User can add more.

```
id          TEXT PK
name        TEXT
default_step_count  INTEGER (1, 2, or 3)
notes       TEXT NULL
created_at  INTEGER
updated_at  INTEGER
deleted_at  INTEGER NULL
```

5.2.3 `account_templates`

Reusable templates for account configurations (e.g., "2-step 5K 4%DD/8%Total").

id	TEXT PK
name	TEXT
prop_firm_id	TEXT FK → prop_firms.id
step_count	INTEGER
account_size_cents	INTEGER
leverage	INTEGER (e.g., 30, 50, 100)
daily_drawdown_type	TEXT ('percent_of_balance' 'percent_of_equity' 'fixed_amount')
daily_drawdown_value	INTEGER (basis points if percent, cents if fixed)
total_drawdown_type	TEXT (same enum)
total_drawdown_value	INTEGER
drawdown_basis	TEXT ('initial_balance' 'high_water_mark' 'previous_day_close')
profit_target_phase_1_pct	INTEGER (basis points, e.g., 800 = 8%)
profit_target_phase_2_pct	INTEGER NULL
profit_target_phase_3_pct	INTEGER NULL
min_trading_days	INTEGER NULL
max_trading_days	INTEGER NULL
weekend_holding_allowed	INTEGER (0/1)
news_trading_allowed	INTEGER (0/1)
consistency_rule_pct	INTEGER NULL (e.g., max single-day profit as % of total)
notes	TEXT NULL
is_archived	INTEGER (0/1)
created_at	INTEGER
updated_at	INTEGER

5.2.4 **accounts**

Each challenge attempt = one account record.

id	TEXT PK
display_name	TEXT (user-assigned, e.g., "Attempt 47")
template_id	TEXT FK → account_templates.id NULL (nullable if
fully custom)	
prop_firm_id	TEXT FK → prop_firms.id
step_count	INTEGER
current_phase	INTEGER (1, 2, 3, or 0 for funded)
account_size_cents	INTEGER
leverage	INTEGER
daily_drawdown_type	TEXT
daily_drawdown_value	INTEGER
total_drawdown_type	TEXT
total_drawdown_value	INTEGER
drawdown_basis	TEXT
profit_target_pct	INTEGER (for current phase)
min_trading_days	INTEGER NULL
max_trading_days	INTEGER NULL
weekend_holding_allowed	INTEGER
news_trading_allowed	INTEGER
consistency_rule_pct	INTEGER NULL
challenge_cost_cents	INTEGER (fee paid for this account)
start_date	INTEGER
status	TEXT ('active' 'passed' 'failed' 'paused'
'retired')	
end_date	INTEGER NULL
end_reason	TEXT NULL
peak_equity_cents	INTEGER
current_equity_cents	INTEGER
notes	TEXT NULL
created_at	INTEGER

updated_at	INTEGER
deleted_at	INTEGER NULL

5.2.5 `account_rules`

Per-account customizable trading rules (user's own rules, separate from firm rules).

id	TEXT PK
account_id	TEXT FK → accounts.id
rule_key	TEXT (see enum below)
enabled	INTEGER (0/1)
value	TEXT (JSON – rule-specific config)
priority	INTEGER (display order)
created_at	INTEGER
updated_at	INTEGER

Built-in rule keys (seeded per account):

- `max_trades_per_day` → `{ value: 2 }`
- `max_risk_per_trade_pct` → `{ value_bps: 100 }` (1%)
- `max_daily_loss_pct` → `{ value_bps: 200 }` (2%)
- `min_rr_ratio` → `{ value: 2.0 }`
- `require_mss_confirmation` → `{ enabled: true }`
- `require_invalidation_text` → `{ enabled: true, min_chars: 20 }`
- `no_sl_widening` → `{ enabled: true }`
- `cooldown_after_loss_minutes` → `{ value: 30 }`
- `daily_stop_after_losses` → `{ value: 2 }` (stop after N consecutive losses)
- `killzone_only` → `{ enabled: true, zones: ["london", "ny_am"] }`

- `require_hft_bias_logged` → `{ enabled: true }`
- `max_overall_daily_loss_hard_stop_pct` → `{ value_bps: 300 }` (3% hard lock)

All user-editable in Settings > Rules per account. All enforced by the rules engine.

5.2.6 `pairs`

User-customizable instruments.

```
id          TEXT PK
symbol      TEXT UNIQUE (e.g., "EURUSD")
display_name TEXT (e.g., "EUR/USD")
asset_class TEXT ('forex' | 'indices' | 'commodities' | 'crypto' |
'stocks' | 'other')
pip_decimal INTEGER (4 for EURUSD, 2 for JPY pairs, etc.)
pip_value_per_standard_lot_cents INTEGER (e.g., 1000 for EURUSD = $10/pip ×
100 for cents)
correlated_with TEXT NULL (JSON array of symbols for SMT, e.g., ["GBPUSD",
"USDXX"])
active      INTEGER (0/1)
display_order INTEGER
notes      TEXT NULL
created_at  INTEGER
updated_at  INTEGER
```

Seeded with: EURUSD, GBPUSD, USDJPY, AUDUSD, USDCAD, USDCHEF, NZDUSD, XAUUSD, US30, NAS100, SPX500, BTCUSD. User can add/edit/archive.

5.2.7 `setups`

User-customizable setup types.

```
id          TEXT PK
name        TEXT (e.g., "FVG", "OB", "Breaker", "Silver Bullet")
category    TEXT (e.g., "ICT", "SMC", "Custom")
description  TEXT NULL
color       TEXT (hex, for chart/tag display)
active      INTEGER (0/1)
display_order INTEGER
created_at  INTEGER
updated_at  INTEGER
```

Seeded with: FVG, Order Block, Breaker Block, Mitigation Block, Judas Swing, SMT Divergence, Silver Bullet, Liquidity Sweep Reversal, Turtle Soup, Power of Three.

5.2.8 **killzones**

User-customizable time windows.

```
id          TEXT PK
name        TEXT (e.g., "London", "NY AM", "Silver Bullet AM")
start_time_utc TEXT (HH:MM format)
end_time_utc TEXT
color       TEXT (hex)
active      INTEGER (0/1)
display_order INTEGER
notes       TEXT NULL
created_at  INTEGER
updated_at  INTEGER
```

Seeded with: Asia (00:00-05:00 UTC), London (07:00-10:00 UTC), NY AM (12:00-15:00 UTC), NY PM Silver Bullet (19:00-20:00 UTC), London Silver Bullet (09:00-10:00 UTC). Times are UTC; display converted to user's timezone.

5.2.9 sessions

A trading session = one session of pre-market bias logging. Typically one per day.

```
id                TEXT PK
account_id        TEXT FK → accounts.id
session_date      TEXT (YYYY-MM-DD)
daily_bias        TEXT ('bullish' | 'bearish' | 'neutral')
daily_bias_reason TEXT
h4_bias           TEXT
h4_bias_reason    TEXT
h1_bias           TEXT
h1_bias_reason    TEXT
htf_liquidity_target TEXT NULL
dxy_bias          TEXT NULL ('bullish' | 'bearish' | 'neutral' | 'n/a')
smt_notes         TEXT NULL
session_plan      TEXT NULL (free text – what am I looking to do today)
key_levels        TEXT NULL (JSON array of price levels)
created_at        INTEGER
updated_at        INTEGER
locked_at         INTEGER NULL (once set, session context can't be
retroactively edited)
```

5.2.10 trades

The core record.

```

id TEXT PK
account_id TEXT FK → accounts.id
session_id TEXT FK → sessions.id NULL
pair_id TEXT FK → pairs.id
setup_id TEXT FK → setups.id
killzone_id TEXT FK → killzones.id NULL
mode TEXT ('live' | 'sim' | 'backtest')
direction TEXT ('long' | 'short')
status TEXT ('planned' | 'open' | 'closed' |
'cancelled')

-- Plan (filled pre-trade)
entry_price INTEGER (price × 10^pip_decimal)
stop_loss_price INTEGER
take_profit_price INTEGER
sl_pips INTEGER (computed, tenths)
rr_ratio INTEGER (×100, so 2.0 = 200)
lot_size INTEGER (×100, so 0.50 lots = 50)
risk_amount_cents INTEGER
risk_pct_bps INTEGER (basis points)
planned_invalidation TEXT (required, min 20 chars)

-- Context
mss_confirmed INTEGER (0/1)
htf_bias_aligned INTEGER (0/1)
dxy_aligned INTEGER NULL (0/1)
smt_confirmed INTEGER NULL (0/1)
correlated_pair_used TEXT NULL

-- Emotional state (pre-trade)
pre_calm_score INTEGER (1-10)
pre_urgency_score INTEGER (1-10)

```

```

pre_need_score                INTEGER (1-10)

-- Execution (filled post-trade)
actual_entry_price            INTEGER NULL
actual_entry_time             INTEGER NULL
exit_price                    INTEGER NULL
exit_time                     INTEGER NULL
exit_reason                    TEXT NULL ('tp' | 'sl' | 'manual' | 'be' |
'partial_full' | 'timeout')
pnl_cents                     INTEGER NULL
pnl_r                         INTEGER NULL (×100)
pnl_pct_bps                   INTEGER NULL
max_drawdown_during_trade_pct_bps INTEGER NULL
mae_pips                      INTEGER NULL (max adverse excursion)
mfe_pips                      INTEGER NULL (max favorable excursion)
duration_minutes              INTEGER NULL

-- Honesty & rules
followed_plan_exactly          INTEGER NULL (0/1)
plan_changes_description        TEXT NULL
sl_moved                      INTEGER NULL (0/1)
sl_moved_reason                TEXT NULL
tp_moved                      INTEGER NULL (0/1)
entered_before_mss             INTEGER NULL (0/1)
revenge_trade_flag             INTEGER NULL (0/1, auto-computed if entered
within cooldown window after loss)
rules_broken                   TEXT NULL (JSON array of rule_key values)
is_clean                       INTEGER NULL (0/1, computed: rules_broken is
empty AND followed_plan_exactly)

-- Post-trade reflection
post_calm_score                INTEGER NULL (1-10)

```

```

what_i_did_right      TEXT NULL
what_i_did_wrong      TEXT NULL
tags                  TEXT NULL (JSON array)

-- v2 prep (unused in v1, set to NULL)
broker_source         TEXT NULL
broker_trade_id       TEXT NULL
imported_at           INTEGER NULL

created_at            INTEGER
updated_at            INTEGER
deleted_at            INTEGER NULL

```

5.2.11 `trade_screenshots`

```

id                    TEXT PK
trade_id              TEXT FK → trades.id
kind                  TEXT ('htf_context' | 'entry' | 'exit' | 'review' | 'other')
filename              TEXT (path relative to screenshots/ folder)
caption               TEXT NULL
created_at            INTEGER

```

5.2.12 `trade_partials`

For v1, user can log partials even though plan is "no partials" — some traders will, some won't.

id	TEXT PK
trade_id	TEXT FK → trades.id
price	INTEGER
lots_closed	INTEGER (×100)
pnl_cents	INTEGER
reason	TEXT NULL
closed_at	INTEGER

5.2.13 **rule_violations**

Historical log of every attempted or committed rule violation.

id	TEXT PK
account_id	TEXT FK → accounts.id
trade_id	TEXT FK → trades.id NULL
rule_key	TEXT
severity	TEXT ('warning' 'blocking' 'logged')
outcome	TEXT ('blocked' 'user_override' 'logged_post_hoc')
context_json	TEXT (JSON – what the rule config was, what the user tried)
created_at	INTEGER

5.2.14 **reviews**

Weekly/monthly review entries. Mandatory weekly ritual (v2).

```

id                TEXT PK
account_id        TEXT NULL (null = cross-account review)
period_type       TEXT ('weekly' | 'monthly' | 'ad_hoc')
period_start      TEXT (YYYY-MM-DD)
period_end        TEXT
top_mistakes      TEXT (JSON array of strings)
best_trade_id     TEXT NULL FK
worst_trade_id    TEXT NULL FK
lesson_next_period TEXT
rule_focus        TEXT NULL (rule_key to focus on next period)
adherence_score   INTEGER (0-100)
notes            TEXT NULL
created_at        INTEGER

```

5.2.15 **cooldowns**

Active cooldowns (tilt detection).

```

id                TEXT PK
account_id        TEXT FK → accounts.id
reason            TEXT ('post_loss' | 'consecutive_losses' | 'daily_loss_hit' |
'rule_violation' | 'manual')
started_at        INTEGER
expires_at        INTEGER
cleared_at        INTEGER NULL
cleared_by        TEXT NULL ('timer' | 'user_acknowledgment')
acknowledgment_text TEXT NULL

```

5.2.16 **backup_log**

```
id          TEXT PK
kind        TEXT ('auto_local' | 'manual' | 'scheduled_cloud')
destination TEXT
filesize_bytes INTEGER
status      TEXT ('success' | 'failed')
error_message TEXT NULL
created_at  INTEGER
```

5.3 Indexes

```
sql

CREATE INDEX idx_trades_account_date ON trades(account_id, created_at);
CREATE INDEX idx_trades_session ON trades(session_id);
CREATE INDEX idx_trades_status ON trades(status);
CREATE INDEX idx_trades_mode ON trades(mode);
CREATE INDEX idx_trades_is_clean ON trades(is_clean);
CREATE INDEX idx_rule_violations_account ON rule_violations(account_id, created_at);
CREATE INDEX idx_sessions_account_date ON sessions(account_id, session_date);
CREATE INDEX idx_cooldowns_active ON cooldowns(account_id, expires_at, cleared_at);
```

5.4 Migration Strategy

- Use Drizzle Kit for migrations.
- Each migration numbered, committed, never edited.
- On app launch, `db.migrate()` runs pending migrations automatically.
- Before migrations run, a snapshot backup is created in `backups/pre-migration/`.
- If migration fails, app refuses to launch and offers restore-from-snapshot UI.

5.5 Seed Data

On first launch:

1. Insert default `settings` (theme = system, timezone = OS detected, etc.).
 2. Insert default `pairs`, `setups`, `killzones`.
 3. Insert "Custom" `prop_firm`.
 4. Trigger first-run onboarding wizard.
-

6. RULES ENGINE

[SPLIT → docs/rules-engine.md]

6.1 Overview

The rules engine is the most important subsystem. It enforces discipline by evaluating rules at three checkpoints:

1. **Pre-trade gate** — before a trade can be submitted.
2. **Trade-modification gate** — when user tries to move SL/TP, add size, etc.
3. **Session gate** — account-level checks (daily loss, cooldowns, etc.).

6.2 Rule Types

Each rule implements this interface:

```
typescript
```



```
interface Rule {
  key: string;           // e.g., 'max_trades_per_day'
  label: string;         // UI-displayed human name
  description: string;   // What it does, in plain English
  category: 'risk' | 'process' | 'timing' | 'behavior';
  severity: 'blocking' | 'warning' | 'logged';
  defaultConfig: Record<string, unknown>;
  configSchema: ZodSchema; // For the settings UI
  evaluate(context: RuleContext): RuleEvaluation;
}

interface RuleContext {
  account: Account;
  accountRules: AccountRuleConfig[];
  currentSession: Session | null;
  tradeInProgress?: PartialTrade;
  tradesToday: Trade[];
  recentTrades: Trade[]; // last N trades
  now: number;           // timestamp
  activeCooldowns: Cooldown[];
}

interface RuleEvaluation {
  passed: boolean;
  severity: 'blocking' | 'warning' | 'info';
  message: string; // Shown to user
  details?: string; // Expanded explanation
  canOverride: boolean; // Can user force-override with typed acknowledge
  suggestedAction?: string; // e.g., "Wait 12 more minutes."
}
```

6.3 Built-in Rules (v1)

Every rule is independently enable/disable-able per account.

Risk Rules:

- `max_risk_per_trade_pct` — computes risk from SL distance and lot size; blocks if >configured %.
- `max_daily_loss_pct` — sum of today's closed P&L; if >limit, session blocks.
- `max_overall_daily_loss_hard_stop_pct` — hard lock; even overrides disabled.
- `min_rr_ratio` — blocks if RR < configured.
- `position_size_matches_plan` — warns if actual lot size differs from calculated.

Process Rules:

- `require_mss_confirmation` — MSS checkbox must be checked pre-trade.
- `require_invalidation_text` — planned_invalidation must be filled with min chars.
- `require_htf_bias_logged` — session must have bias logged before any trade.
- `no_sl_widening` — if user tries to move SL against position, block.
- `require_killzone` — blocks entries outside configured killzones.
- `require_dxy_check` — warns if DXY bias not logged in session.

Behavior Rules:

- `max_trades_per_day` — hard cap.
- `cooldown_after_loss_minutes` — blocks new entries for N minutes after a loss.
- `daily_stop_after_losses` — session auto-locks after N consecutive losses.
- `no_revenge_trade_window` — flags/blocks trades taken within X min of a loss on same pair.

- `emotional_state_gate` — warns if `pre_urgency > 7` or `pre_need > 6`, with typed acknowledgment to proceed.

Timing Rules:

- `weekend_holding_blocked` — blocks open trades into weekend if firm doesn't allow.
- `news_window_blocked` — v2 feature placeholder (disabled in v1).
- `min_trading_days_check` — shows progress toward min trading days for phase.

6.4 Evaluation Flow

Pre-trade submit flow:

1. Gather RuleContext.
2. Run all enabled rules in priority order.
3. Collect all evaluations.
4. If any blocking rule fails:
 - a. Show blocking modal listing failed rules.
 - b. If `canOverride` on all failed rules, show override UI (typed acknowledgment required).
 - c. If any rule is non-overrideable (e.g., hard DD), no override possible.
5. If warnings only, show warning modal with "Proceed" / "Cancel" options.
6. If all passed, proceed to submit.

All evaluations are logged in `rule_violations` **even if the rule passed-with-warning or user overrode. This builds the behavioral dataset.**

6.5 Cooldown System

When triggered:

- Creates a row in `cooldowns` with `expires_at`.

- UI shows persistent countdown banner with minutes remaining.
- New trade submissions blocked until cleared.
- Cleared by: (a) timer expires, (b) user types the exact phrase "I am trading my plan, not my emotions" for the consecutive-losses cooldown (v2).

6.6 Override System

For rules with `canOverride: true`:

- Modal shows: rule name, why it's firing, current value vs. threshold.
- User must **type** (not click) the word `OVERRIDE`.
- A reason (min 30 chars) is required.
- Override is logged as a `rule_violation` with `outcome: 'user_override'`.
- Heavy visual feedback (crack animation on relevant UI elements).
- The trade is tagged `is_clean: false` permanently.

6.7 Hard Locks (Non-overrideable)

- Maven-style max-daily-drawdown hard stop.
- Post-violation 24-hour cooldown (v2).
- System locks that require app restart or cooldown expiry cannot be bypassed.

6.8 Session Lock State

The app has a global session state:

- `idle` — no session started today.
- `active` — session started, trades can be taken.
- `paused` — manually paused by user.

- `locked` — rule-triggered lock (e.g., daily loss hit). Only read-only views available until reset time.

Locked state visibly changes the UI:

- Dashboard shows a large "Session Locked" state.
 - Trade entry disabled everywhere.
 - Message explains which rule triggered it and when it unlocks.
-

7. V1 FEATURES — FULL SPEC

`[SPLIT → docs/features-v1.md]`

7.1 Onboarding (First Launch)

Flow:

1. Welcome screen. "Cairn helps you trade the plan, not the emotion." Continue.
2. Create first prop firm (or skip — "Custom" exists). Name field, step count default.
3. Create first account template. Walk through size, DD rules, profit target. Optional.
4. Create first account (or defer). Asks: display name, template (if any), challenge cost.
5. Review default pairs/setups/killzones. Offer to customize now or later.
6. Choose theme (system/light/dark).
7. Choose backup folder. "Where should Cairn back up your data?" Default: `<Documents>/Cairn Backups`. Option to point at Google Drive/Dropbox folder.
8. Done. Land on Dashboard.

Stored in settings: `onboarding_completed: true`.

7.2 Dashboard (Main View)

Layout (top to bottom):

Row 1 — Hero stats strip (4 cards, responsive collapse):

- Discipline Ring (§4.9) — center-left, largest.
- Active Account summary — current equity, distance to DD, distance to profit target, phase progress bar.
- Today — trades taken, P&L, rules broken count, session state.
- Rolling expectancy — R-multiple expectancy over last 20 trades, with mini sparkline.

Row 2 — Quick actions:

- "Log Session Bias" (if today's session not started)
- "New Trade" (only clickable if session active and no locks)
- "Review Last Trade" (opens most recent closed trade)
- "Open Playbook" (opens setups reference)

Row 3 — Recent trades table — last 10, with inline clean/dirty badge, setup, RR, P&L. Click to open full trade detail.

Row 4 — Session context card — today's bias, DXY, key levels, if session logged. Editable until first trade is placed (then locked).

Row 5 — Rule adherence preview — small widget showing rule-break frequency this week.

All numbers animate in with count-up on first render.

7.3 Session Bias Entry

Modal or dedicated page. Required before first trade of the day (if

`require_htf_bias_logged` rule is on).

Fields:

- Daily bias: Bullish / Bearish / Neutral (3 buttons, visually distinct)
- Daily reason: 1-line text (required)
- 4H bias: same
- 4H reason
- 1H bias: same
- 1H reason
- HTF liquidity target: text (optional but encouraged)
- DXY bias: Bullish / Bearish / Neutral / N/A
- SMT pair + notes (optional)
- Session plan: multi-line text (optional)
- Key levels: tag input, user types prices (e.g., `1.0842`, `1.0895`)

Lock behavior: Once the first trade is submitted for this session, the session bias becomes read-only. Edit button replaced with "Locked at [time]". This prevents retroactive narrative editing.

7.4 New Trade — Pre-Trade Flow

This is the most-used flow and must feel fast.

Structure: Right-side slide-over panel (420px wide), opens over dashboard. Escape or click-outside cancels with confirm if any data entered.

Sections (top to bottom, all visible, no tabs):

Context (auto-filled, read-only):

- Account (with phase badge)
- Session bias summary (one line)
- Trade # today (e.g., "Trade 2 of 2 max")
- Current daily P&L
- Active cooldowns (if any — blocks save button)

Instrument & Setup:

- Pair dropdown (searchable, shows recent first)
- Setup dropdown
- Killzone (auto-detected by current time, overridable)
- Mode (Live / Sim / Backtest) — defaults based on account type

Direction:

- Long / Short toggle (large, clear)

Prices (three numeric inputs, JetBrains Mono):

- Entry
- Stop Loss
- Take Profit

Auto-computed (read-only, updates live as prices change):

- SL in pips
- TP in pips
- Risk/Reward ratio (color-coded: red if <1.5, amber 1.5-2, green ≥2)
- Lot size (calculated from risk %)

- Risk in \$ / %

Confluence checkboxes (one-tap each):

- ☐ MSS confirmed
- ☐ HTF bias aligned (auto-checked if direction matches session bias, user can uncheck)
- ☐ DXY aligned
- ☐ SMT confirmed (and a "with pair" sub-select)

Invalidation (required textarea):

- "This trade is wrong if: ____"
- Min 20 characters. Char count shown.
- Cannot submit without filling.

Emotional state (3 sliders, 1-10):

- Calm
- Urgency
- Need-this-to-work

Rule check panel (bottom, always visible):

- Live-updating as fields change.
- Shows each relevant rule with ✓ pass / ⚠ warning / ✗ block.
- If anything blocks: "CANNOT SUBMIT" button (disabled, red).
- If clear: "READY TO TRADE" button (enabled, accent A).

Submit flow:

- Click "Ready to Trade" → trade status = `planned`.
- Opens a confirmation mini-modal: "Order placed in broker? Once clicked, the session bias will lock and trade will be marked open." Two buttons: "Yes, it's placed" or "Actually, cancel".
- On "Yes" → trade status = `open`. Session locks. Dashboard updates.

Save-as-draft option: user can save incomplete trade as `planned` status and come back. Useful when watching for a setup that hasn't fully formed.

7.5 Trade Modification

When a trade is `open`:

- Detail view shows "Move SL" / "Move TP" / "Close Manually" / "Log Partial" actions.
- Moving SL against position triggers `no_sl_widening` rule.
- All modifications logged; `sl_moved`, `tp_moved` flags set.

7.6 Post-Trade Log

When marking a trade closed (from detail view or from a "Close Trade" button on the open-trades list):

Fields:

- Exit price (numeric)
- Exit time (auto = now, editable)
- Exit reason (dropdown: TP / SL / Manual / BE / Timeout / Partial-then-full)
- Max adverse excursion pips (optional)
- Max favorable excursion pips (optional)

Auto-computed: P&L in \$, R, %, duration.

Honesty section (required):

- "Did I follow my plan exactly?" Yes/No radio.
- If No: "What did I change?" textarea.
- "Did I move SL?" Yes/No. If Yes: reason textarea.
- "Did I enter before MSS confirmed?" Yes/No.

Rules broken checklist:

- All account rules listed with checkbox. User ticks those they broke. (The engine also auto-detects several; user can add more.)

Post-trade reflection:

- Post-close calm score (1-10).
- "What I did right" (one sentence).
- "What I did wrong" (one sentence).
- Tags (free tag input).

Screenshots:

- Drag-drop or file picker. Up to 4 screenshots per trade (HTF context, entry, exit, review).
- Stored in `screenshots/<trade_id>/`.

Submit: Trade becomes `closed`. `is_clean` computed. Dashboard updates. Toast confirms.

7.7 Trade Log (History)

Full-screen table view with filters.

Filters (left sidebar or top bar):

- Date range
- Account
- Pair
- Setup
- Killzone
- Mode (live/sim/backtest)
- Clean vs Dirty
- Win/Loss/BE
- Tags

Columns (sortable, user-configurable):

- Date/Time, Pair, Setup, Killzone, Direction, Entry, SL, TP, RR, Risk%, Result, P&L \$, P&L R, P&L %, Clean?, Rules Broken count, Duration

Row actions: View details, Duplicate as planned trade, Add note, Delete (soft delete).

Bulk: Export selected to CSV.

7.8 Trade Detail View

Click any trade → full view with:

- All fields in an organized layout.
- Screenshots gallery with lightbox.

- Rule evaluations that fired at submit time.
- Related trades (same day).
- Edit / Delete / Export.

7.9 Analytics (See §8 for detail)

Full analytics dashboard with 6 tabs. See Analytics section.

7.10 Accounts

List view of all accounts. Each shows:

- Display name, firm, size, phase, status.
- Current equity vs starting.
- DD used / available.
- Profit target progress.
- Days alive.
- Trades taken on this account.
- Clean rate on this account.

Actions per account: View dashboard (switches context), Edit, Mark as Passed, Mark as Failed (requires reason), Archive.

Create new account: uses template or custom.

7.11 Templates

Create, edit, archive account templates. Templates pre-fill new accounts quickly.

7.12 Settings

Tabbed:

General:

- Theme, language (English only v1), timezone, week starts on, number formatting region.

Rules (per account, with global defaults):

- Each built-in rule with toggle + config editor.
- Reset to defaults.
- Export/import rule profile (JSON).

Pairs:

- Manage list, add custom, edit pip decimal, reorder, archive.

Setups:

- Manage list, add custom, edit color, reorder, archive.

Killzones:

- Manage list, edit times, reorder, archive.

Prop Firms:

- Manage firms, add custom.

Account Templates:

- Manage templates.

Backups:

- Set backup folder.
- Set schedule (daily at X time, on app close, manual only).
- View backup log.
- Manual backup now.
- Restore from backup.

Data:

- Export all data (JSON/CSV).
- Open data folder (OS file explorer opens app data dir).
- Reset all data (destructive, requires typed confirmation).

About:

- Version, links to docs (local help files), credit line (Jai Akash).

7.13 Help / Playbook

Local markdown viewer with:

- Built-in reference for ICT concepts (user-editable).
- User's own playbook notes.
- Setup examples.
- Not a tutorial — a quick reference for during-trade thinking.

Stored as markdown files in app data folder; rendered with a markdown component.

8. ANALYTICS — DETAILED SPEC

[SPLIT → docs/analytics.md]

8.1 Filter Bar (Persistent)

Every analytics tab respects the global filter bar:

- Date range (presets: Today, Last 7d, Last 30d, This Month, Last Month, All Time, Custom)
- Account (All / specific)
- Mode (Live / Sim / Backtest / All)
- Pair (multi-select)
- Setup (multi-select)
- Killzone (multi-select)
- Clean trades only (toggle)

Filters persist across tabs within a session.

8.2 Tab 1 — Performance

Top stat row:

- Total trades, Win rate, Expectancy (R), Total R, Net P&L \$, Net P&L %, Profit factor.

Equity curve chart:

- Line chart showing cumulative P&L over the filtered range.
- Toggle: in \$ or in R.
- Overlays: max drawdown shaded region; account DD limit horizontal line.

Distribution chart:

- Histogram of R-multiples per trade.
- Color: red for losses, green for wins, gold bucket for break-even.

Streak card:

- Current streak (win/loss, count).
- Longest winning streak, longest losing streak.

Daily breakdown heatmap:

- Calendar-style heatmap showing daily P&L. Darker green = better day, darker red = worse.

8.3 Tab 2 — Rule Adherence (Primary Tab)**Hero metric:**

- Rule Adherence Score % (large, center). Target: 95%+.
- Trend arrow vs previous period.

Clean vs Dirty split:

- Two cards side-by-side:
 - Clean trades: count, win rate, expectancy, total R.
 - Dirty trades: same stats.
- The difference between these two is often the most revealing number.

Top rules broken (bar chart):

- Horizontal bars showing frequency of each rule violation.
- Hover shows: when broken, resulting avg P&L.

Rule-break impact table:

- For each rule: # times broken, net P&L when broken, win rate when broken.
- Sorted by net impact.

Blocked trades counter:

- How many trades the engine prevented this period.
- What would their projected impact have been (estimated from past dirty-trade data).

Adherence trend line:

- Weekly adherence % over time.

8.4 Tab 3 — Setup Performance

Setup × session matrix:

- Grid: rows = setups, columns = killzones. Cell value = expectancy or win rate (toggle).
- Color-coded.

By setup:

- Bar chart: win rate by setup.
- Expectancy by setup.
- Avg RR achieved by setup.

By day of week:

- Win rate and expectancy per day.

With MSS vs without MSS:

- Side-by-side comparison cards.

DXY aligned vs against:

- Side-by-side comparison.

SMT confirmed vs not:

- Side-by-side.

8.5 Tab 4 — Behavioral

Emotional state buckets:

- Three cards: Calm (1-4 urgency), Neutral (5-7), Urgent (8-10).
- Each shows: # trades, win rate, expectancy.

Need-to-work score bucket:

- Same structure.

Post-loss behavior:

- First trade after a loss: win rate, expectancy.
- Second trade after a loss: win rate, expectancy.
- Revenge trades (flagged): separate card.

Trade # of day:

- Win rate/expectancy for trade 1, trade 2, trade 3+.
- Expected insight: trade 3+ is negative expectancy.

Time of day heatmap:

- Hourly grid × day of week. Expectancy per cell. Color-coded.

Loss-then-win pattern tracker:

- What % of recoveries are clean trades vs dirty.

8.6 Tab 5 — Accounts & Phases

Account ladder:

- Vertical list of all accounts, chronologically.
- Each row: name, firm, size, phase progression, days alive, outcome, cost.
- Clickable → account detail view.

Phase pass rate trend:

- Line chart: rolling Phase 1 pass rate, Phase 2 pass rate, overall funded rate.

Cost analysis:

- Total \$ spent on challenge costs.
- Total \$ earned from payouts (user-logged).
- Net position.
- Cost per trade across history.

Cause of failure breakdown:

- Pie chart: which rule was broken most on failed accounts.

Days to failure distribution:

- Histogram: how many days accounts typically survive before failing.

Pattern insight cards:

- Auto-generated insight text cards like:
 - "70% of your blown accounts broke `max_trades_per_day` rule within 48h of failure."
 - "Your Phase 2 failures have clean rate of 62%. Phase 2 successes: 94%."

These are computed from queries and displayed when statistically meaningful (min sample size).

8.7 Tab 6 — Review

- Weekly review form (v2 mandatory gate, v1 optional).
- List of past reviews.
- Monthly summary preview.
- Export review as PDF (v2).

8.8 Chart Standards

- Recharts throughout.
- Colors from design system only.
- No gridlines unless necessary; use subtle horizontal only.
- Tooltips: custom-styled, dark/light aware.

- Empty states: descriptive text + suggestion, never blank charts.
 - Min data thresholds: hide or caveat stats with <10 trade samples.
-

9. UI FLOWS — KEY JOURNEYS

[SPLIT → docs/ui-flows.md]

9.1 Start-of-Day Flow

1. Open Cairn.
2. Dashboard shows "No session logged today" banner.
3. Click "Log Session Bias."
4. Fill bias, key levels, plan. Save.
5. Session active. Dashboard updates.

9.2 Trade Flow (Happy Path)

1. Click "New Trade."
2. Fill pair, setup, prices, invalidation, emotional state.
3. Rule panel shows all green.
4. Click "Ready to Trade."
5. Confirm "Order placed in broker." Session locks.
6. Later, click trade → "Close Trade."
7. Fill exit price, reflection.
8. Trade closed. Dashboard updates.

9.3 Trade Flow (Rule Block Path)

1. Click "New Trade."
2. User has already taken 2 trades today.
3. `max_trades_per_day` rule blocks immediately on panel open.
4. Rule panel shows red: "Trade limit reached. 2/2 today."
5. Submit button disabled.
6. User can: close panel, or click "Request Override" (if rule allows).
7. Override requires typing OVERRIDE + reason.
8. If overridden, trade proceeds but is pre-flagged dirty.

9.4 Post-Loss Tilt Flow

1. User closes a losing trade.
2. Cooldown created (30 min default).
3. Dashboard shows banner: "Cooldown active. New trades blocked for 28:42."
4. If user tries "New Trade": modal blocks with countdown.
5. After cooldown expires, banner clears.
6. If user hits consecutive loss threshold: longer lock, requires typed acknowledgment.

9.5 Weekly Review Flow (v1 optional)

1. Sunday: dashboard shows "Weekly Review pending" card.
2. User clicks → Review form opens.
3. Fill top 3 mistakes, best trade, worst trade, lesson, rule focus.
4. Save. Review stored.
5. Next week's rule focus surfaces on dashboard as reminder.

9.6 New Account Flow

1. Accounts page → "New Account."
2. Choose template or custom.
3. Fill fields.
4. Review rules (all defaults inherited, user can edit now).
5. Save. Account becomes active account.

9.7 Backup Flow

- Automatic: daily at configured time, also on app close if enabled.
- Manual: Settings → Backups → "Backup Now" → confirm destination.
- Restore: Settings → Backups → "Restore from Backup" → pick zip → confirm (destructive modal) → app restarts.

9.8 First Launch

See §7.1.

9.9 Migration-Failed State

- On migration failure: app shows "Migration failed" screen.
 - Offers: "Restore pre-migration snapshot" (recommended), "Try again", "View error log", "Contact (copy error)".
 - No normal UI accessible until resolved.
-

10. CUSTOMIZATION

[SPLIT → docs/customization.md]

Everything that could reasonably vary per user or per account must be customizable without code changes.

10.1 What's Customizable

Global:

- Pairs (add, edit, archive)
- Setups (add, edit, archive)
- Killzones (times, names)
- Prop firms
- Account templates
- Theme, timezone, formatting

Per account:

- Every rule (enable, config value)
- Account-specific rule overrides
- Display name

Per session:

- Bias fields
- Key levels

10.2 Adding a Custom Pair

Form:

- Symbol (unique, uppercase, no spaces)
- Display name
- Asset class
- Pip decimal (how many decimals = 1 pip)
- Pip value per standard lot (\$ per pip at 1.0 lot)
- Correlated symbols (for SMT)

10.3 Adding a Custom Setup

- Name, category, description, display color, active flag.

10.4 Adding a Custom Rule (v2)

Built-in rules are the initial set. v2 will allow user-defined simple rules via a rule builder (e.g., "Block if I'm trading on a Friday afternoon"). Out of scope for v1.

10.5 Account Template Customization

All fields in §5.2.3 are user-editable. Templates can be exported/imported as JSON.

11. FUTURE INTEGRATIONS (V2)

[SPLIT → docs/integrations-future.md]

11.1 MT5/cTrader Integration Architecture

v1 preparation:

- `trades.broker_source`, `trades.broker_trade_id`, `trades.imported_at` exist from day 1.
- `electron/services/import-adapters/` folder exists with `manual.ts` placeholder.

v2 adapter interface:

typescript

```
interface BrokerAdapter {  
  name: string;  
  authenticate(config: Record<string, unknown>): Promise<AuthResult>;  
  fetchTrades(since: number): Promise<BrokerTrade[]>;  
  mapToTrade(brokerTrade: BrokerTrade, context: ImportContext): Partial<Trade>;  
  supportsLiveStream: boolean;  
  streamTrades?(onTrade: (t: BrokerTrade) => void): Disposable;  
}
```

Adapters to build (v2):

- MT5 (via Python bridge or REST API exposed by MT5 terminal)
- cTrader (via cTrader Open API)
- TradeLocker
- Match-Trader

Import flow:

- User connects account via Settings → Integrations.
- Adapter imports past trades (filtered to after a chosen date).
- Imported trades show "Broker import" badge; honesty fields and reflection still filled manually.

- Live stream: new broker trades auto-create planned trades for user to fill context on.

11.2 Other V2 Features (From §7.12 v2 list)

- Killzone heatmap (specified in §8.5; unlock full version in v2)
- Tilt detection expansions (24h auto-locks, consecutive-loss phrases)
- Pre-market bias journal expansion
- Backtest mode enhancements
- Mandatory weekly review (gating UI until completed)
- Screenshot flashcard review mode
- Monthly PDF export
- Goal/guardrail progress bars (detailed)
- User-defined custom rules builder

11.3 Never-Features (Intentionally Out of Scope)

- Social/community features
- Copy trading
- Broker order execution from Cairn
- News integration (explicitly descoped by user)
- AI-generated trade suggestions
- Automated strategy backtesting (different tool class)

12. TESTING

[SPLIT → docs/testing.md]

12.1 Philosophy

Test the things that would be expensive to get wrong. Money math, rules engine, and data integrity are the priority. UI polish can be covered by manual QA.

12.2 Unit Tests (Vitest)

Required coverage:

- All calculators (`lib/calculators.ts`): lot size, pip value, R-multiple, RR, P&L.
- Rules engine: each rule has isolated tests with multiple scenarios.
- Formatters: money, pips, percentages, locale handling.
- Database utilities.

Target: >90% coverage on `lib/`, `services/`, `rules-engine/`.

12.3 Integration Tests

- IPC round-trips with real SQLite.
- Migration runs successfully on fresh and existing DBs.
- Backup creates valid zip; restore recreates identical state.

12.4 E2E Tests (Playwright, Electron)

Critical paths:

- First-launch onboarding completes.
- New trade: happy path.
- New trade: rule-blocked path.
- Trade close + journal entry.
- Dashboard reflects new trade.

- Theme switch persists.
- Backup creation and restore.

12.5 Manual QA Checklist (Pre-release)

Documented checklist covering:

- All forms validate correctly.
 - All rules trigger at correct thresholds.
 - All animations respect reduced-motion.
 - Dark/light both render correctly.
 - Accessibility basics: focus order, ARIA labels, keyboard nav.
-

13. CONVENTIONS

[SPLIT → docs/conventions.md]

13.1 Code Style

- Prettier for formatting (config committed).
- ESLint with strict TypeScript rules.
- No `any`. If unavoidable, `unknown` + zod validation.
- No default exports except React components (single per file).

13.2 Naming

- Files: kebab-case (`trade-entry-panel.tsx`).
- Components: PascalCase (`TradeEntryPanel`).

- Hooks: camelCase starting with `use` (`useActiveAccount`).
- DB columns: snake_case (`entry_price`).
- TypeScript types/interfaces: PascalCase (`TradeDraft`).
- Enums: PascalCase values or string-literal unions (prefer unions).

13.3 Commits

- Conventional Commits (`feat:`, `fix:`, `refactor:`, `chore:`, `docs:`, `test:`).
- Imperative mood, lowercase, no period.
- Reference feature in body when non-trivial.

13.4 Branching

- `main` = stable.
- Feature branches: `feat/<short-name>`.
- No direct commits to main except for trivial fixes.

13.5 Git Hooks

- Pre-commit: `lint-staged` runs Prettier + ESLint on changed files.
- Pre-push: `pnpm typecheck` and `pnpm test:unit`.

13.6 Environment

- No `.env` secrets needed for v1 (everything local).
- `.env.example` committed with any non-secret defaults.
- Platform-specific paths resolved via Node `path` + Electron `app.getPath()`.

13.7 Error Handling

- Every IPC returns `{ ok, data } | { ok, error }`.
- Errors include: `code`, `message`, optional `details`.
- User-facing errors: plain English, never stack traces.
- Unexpected errors in main process: logged to `<data>/logs/` and surfaced in Settings > About > Diagnostics.

13.8 Logging

- `electron-log` for main process.
- Log levels: error, warn, info, debug.
- Rotated daily, max 7 days retained.
- Accessible via Settings for copy/export.

13.9 Performance Budgets

- Cold app launch → dashboard interactive: <1500ms.
- New trade panel open: <120ms.
- Analytics tab switch: <400ms with 1000+ trades.
- Rule evaluation on field change: <30ms.
- SQLite writes for trade submit: <50ms.

13.10 Accessibility

- Keyboard navigation for all interactive elements.
- Focus visible indicators (custom-styled, never removed).
- ARIA roles/labels for complex components.

- Color contrast $\geq 4.5:1$ for body, $3:1$ for UI elements.
- Respect `prefers-reduced-motion` and `prefers-color-scheme`.
- Text resizable up to 150% without layout break.

13.11 Self-Healing / Self-Correcting Practices

Specific requirements Claude Code must implement:

1. **Database integrity check on launch.** Run `PRAGMA integrity_check`. If fails, attempt backup restore prompt.
2. **Pending migration detection.** If DB schema version < code expected, run migrations with snapshot backup first.
3. **Crash log capture.** Uncaught main-process errors write to a crash log with app state summary.
4. **Auto-backup before risky operations.** Before migrations, data reset, bulk imports (v2), an auto-backup runs.
5. **Settings schema validation on load.** Settings are zod-validated; invalid values reset to defaults with a toast notification.
6. **Orphaned file cleanup.** On launch, check `screenshots/` for files with no corresponding trade and move them to `screenshots/_orphaned/`.
7. **Duplicate prevention.** Trade submit checks for near-duplicate (same account, same pair, same direction, entry within 1 pip, within last 5 min) and warns.
8. **Stale lock recovery.** If a cooldown has `expires_at` in the past but `cleared_at` is null on launch, auto-clear it.
9. **Timezone change handling.** If OS timezone changes, app detects on next launch, warns user, offers to update session date boundaries.

10. **Version mismatch guard.** On launch, compare app version with the version that last wrote the DB. Downgrades are blocked with a clear message (upgrades are fine).
11. **Transaction wrappers.** All multi-row writes (trade + violations + screenshots) are wrapped in a single SQLite transaction.
12. **Idempotent IPC.** Create operations accept optional client-generated IDs so retries don't duplicate.

13.12 Development Mindset for Claude Code

Instructions for the agent implementing this spec:

- **Build vertically, not horizontally.** Complete one feature end-to-end (data model → IPC → UI → tests) before starting another. This ensures each shipped feature actually works.
- **Order of build (suggested):**
 1. Project scaffold (Electron + React + Vite + TS + Tailwind + shadcn/ui init).
 2. Database setup with drizzle + first migration + seed.
 3. IPC plumbing with one sample domain (settings).
 4. Design system primitives (colors, fonts, base components).
 5. Shell UI (sidebar, top bar, dark/light toggle, attribution).
 6. Onboarding.
 7. Accounts + templates + prop firms CRUD.
 8. Pairs/setups/killzones CRUD.
 9. Session bias entry.
 10. Rules engine core + built-in rules.
 11. New trade pre-trade panel.
 12. Post-trade log + trade detail.

13. Trade log (history view).
 14. Dashboard.
 15. Analytics (tab by tab).
 16. Settings (tab by tab).
 17. Backup/restore.
 18. Self-healing utilities.
 19. Polish, animations, accessibility pass.
 20. Testing passes.
- **Never skip the rules engine.** It is the heart of the app. If an early feature needs it, stub it properly; do not shortcut it.
 - **When in doubt, re-read §1 and §2.** The philosophy and non-negotiables resolve most design choices.
 - **If a conflict appears between this spec and another instruction, this spec wins.** Surface the conflict to the user rather than silently resolving.
 - **Ship small commits.** Every commit should leave the app runnable.
 - **Write real tests for the rules engine and calculators before writing UI that depends on them.**
 - **Before adding any library not listed in §3.1, pause and justify.** The stack is already chosen; additions dilute it.
-

14. APPENDIX — DECISIONS LOCKED

The following decisions are **locked** and should not be revisited without explicit user confirmation:

1. Electron, not Tauri.

2. Manual entry in v1, broker adapters in v2.
 3. Local SQLite, no cloud database.
 4. Cloud backup via existing desktop sync services (user points to a synced folder), not via cloud APIs.
 5. No news integration, ever (per user direction).
 6. No prop firm names hardcoded in UI.
 7. No user-history references in UI (founding document context only).
 8. App name: **Cairn**.
 9. Attribution: "Designed & built by Jai Akash" in sidebar bottom-left, nowhere else.
 10. Color palette: Graphite & Citrus (dark) / Bone & Forest (light). Zero purple/cyan.
 11. Fonts: Inter + JetBrains Mono.
 12. v1 feature scope as listed in §7. v2 additions in §11.
 13. Rule-gating is blocking by default; overrides require typed acknowledgment; hard locks cannot be overridden.
 14. Prevention over detection is the app's north star.
-

15. GLOSSARY

- **ICT** — Inner Circle Trader methodology.
- **SMC** — Smart Money Concepts.
- **BOS** — Break of Structure.
- **CHoCH** — Change of Character.
- **MSS** — Market Structure Shift (synonym of CHoCH in common usage).
- **FVG** — Fair Value Gap.

- **OB** — Order Block.
 - **OTE** — Optimal Trade Entry.
 - **PD Array** — Premium/Discount Array.
 - **SMT** — Smart Money Technique (divergence between correlated pairs).
 - **Killzone** — Time window with historically high volume/volatility.
 - **Judas Swing** — Initial manipulation move against the eventual true direction.
 - **Silver Bullet** — Specific 1-hour killzone within NY or London.
 - **DD** — Drawdown.
 - **RR** — Risk-to-Reward ratio.
 - **R / R-multiple** — Profit or loss expressed as multiple of initial risk.
 - **MAE** — Maximum Adverse Excursion.
 - **MFE** — Maximum Favorable Excursion.
 - **Clean trade** — Trade with zero rules broken and plan followed exactly.
 - **Dirty trade** — Trade with any rule broken or plan deviation.
 - **DXY** — US Dollar Index.
-

16. END STATE DEFINITION (WHAT "V1 DONE" MEANS)

v1 is complete when:

1. A user can install, onboard, and configure an account in under 10 minutes.
2. A user can log a session bias in under 60 seconds.
3. A user can submit a pre-trade plan with full rule gating in under 60 seconds (with prior context loaded).
4. A user can log a trade close with full reflection in under 60 seconds.

5. Every rule in §6.3 is implemented, configurable, and evaluated correctly.
6. Dashboard renders accurate live stats for the active account.
7. Analytics tabs 1-5 are fully functional with 500+ trade datasets.
8. Backup and restore work cleanly, including cross-device via pen drive.
9. Dark and light modes both ship polished.
10. All motion respects reduced-motion preference.
11. All of §13.11 self-healing behaviors are in place.
12. Critical paths in §12.4 pass E2E.
13. No console errors in normal use.
14. Performance budgets in §13.9 are met.
15. App packages successfully for Windows (NSIS), macOS (DMG), Linux (AppImage).

When all 15 are true, v1 ships. v2 planning begins the same day.

End of CLAUDE.md.

A cairn is a stack of stones placed on a trail to mark the way. Each stone is deliberate. Each stone stays where it's placed.

Build the stack.